AGENCE NATIONALE DES PORTS
PORT DE MOHAMMEDIA

RAPPORT DE PROJET

Supervision d’un switch Cisco

avec Prometheus, Grafana et SNMP

« Voir l’état du réseau en temps réel, plutôt que de l’apprendre quand il tombe. »

Rédigé par	**Marwane Hamouyat**
Mission	**Infrastructure réseau — Supervision**
Structure d’accueil	**Agence Nationale des Ports (ANP), Port de Mohammedia**
Date	**Septembre 2026 – Octobre 2026**
Version	**Public — Document de presentation – Projet N°2**

Version publique — toutes les données présentées proviennent d’un environnement de lab.

Sommaire

Ce rapport raconte, dans l'ordre, comment j'ai construit et vérifié une chaîne de supervision complète. Les captures d'écran servent de preuves : chaque étape importante est montrée, pas seulement racontée.

Section	Ce que vous allez y trouver
Introduction	Le besoin, les objectifs et les mots pour comprendre
1. Préparation	Le lab virtuel, les outils et le plan d'adressage
2. Configuration	Le switch, SNMP et le fichier de Prometheus
3. Collecte	Les données brutes, l'exporter et la cible UP
4. Graphiques finaux	Les dashboards et ce qu'ils racontent
Conclusion	Le bilan, les compétences et les prochaines étapes
Annexe	Les commandes et la configuration utilisées

> **COMMENT LIRE CE RAPPORT**
>
> Vous êtes recruteur ou vous découvrez la supervision ? Lisez l'introduction, regardez les graphiques de la section 4, puis la conclusion. Le reste donne le détail technique.
>
> Vous êtes ingénieur ? Les sections 2 et 3 contiennent les commandes, la configuration et les points de vérification. L'annexe rassemble le mémo.
>
> Les termes techniques sont expliqués à leur première apparition, et un lexique express se trouve à la fin de l'introduction.

Le projet en un coup d'œil

Élément	Détail
Contexte	Stage à l'Agence Nationale des Ports (ANP), Port de Mohammedia
Objectif	Superviser un switch Cisco vIOS en temps réel
Environnement	Lab virtuel Containerlab + Docker, avec 6 conteneurs dont 3 switches Cisco vIOS
Chaîne technique	SNMP → snmp-exporter → Prometheus → Grafana
Fréquence de collecte	Toutes les 15 secondes
Livrables	Lab reproductible, 2 dashboards, ce rapport

Introduction

Un réseau qui fonctionne, on n'y pense pas. Un réseau qui tombe, tout le monde le remarque.

Dans un port, cette différence compte. Les équipes s'appuient sur le réseau pour travailler, échanger des données et suivre l'activité. Quand un lien lâche, il ne suffit pas de le savoir : il faut le voir vite, comprendre d'où vient le problème et garder une trace de ce qui s'est passé.

> **L'ESSENTIEL EN 30 SECONDES**
>
> J'ai monté un lab réseau virtuel avec trois switches Cisco vIOS, puis installé une chaîne de supervision open source. Le switch expose ses informations via SNMP, un exporter les traduit, Prometheus les collecte toutes les 15 secondes, et Grafana les affiche. Résultat : deux dashboards permettent de voir l'état des interfaces et la bande passante en temps réel. Chaque étape a été vérifiée par une capture d'écran.

Contexte et besoin

Pendant mon stage à l'Agence Nationale des Ports (ANP), au Port de Mohammedia, j'ai travaillé sur la supervision d'un switch Cisco vIOS. Un switch, c'est un peu le carrefour du réseau : c'est par lui que passent les échanges entre plusieurs équipements. S'il est mal surveillé, on découvre souvent le problème au moment où un utilisateur appelle.

L'objectif de mon projet était simple à formuler : rendre visible, en temps réel, l'état de ce switch et de ses interfaces. Derrière cette phrase, il y a une chaîne complète à mettre en place : un lab réseau, une configuration SNMP, un outil de collecte, une base de données de métriques et un tableau de bord.

Sans supervision, on travaille en réaction. On sait qu'un équipement est tombé parce que quelqu'un le signale, pas parce qu'on l'a vu venir. Avec supervision, on passe d'une logique de réparation à une logique de surveillance : on voit l'état du réseau, on repère une anomalie, et on agit avant que l'incident ne prenne de l'ampleur.

Objectifs

- **Monter un lab représentatif** avec Containerlab et Docker, sans toucher au réseau de production.
- **Configurer le switch** avec une adresse IP et activer SNMP en lecture seule.
- **Collecter les métriques des interfaces** avec Prometheus et snmp-exporter.
- **Construire deux dashboards Grafana** : état des interfaces et bande passante.
- **Vérifier chaque maillon de la chaîne** et documenter la démarche avec des captures.
- **Rester reproductible** : le lab doit pouvoir être relancé sur une autre machine, avec des outils open source.

Périmètre

Ce projet se concentre sur un switch, **fed-a-anp**, dans un environnement de lab. Il ne couvre pas le déploiement en production, ni la configuration des alertes, ni la supervision de l'ensemble du réseau de l'ANP. C'est une brique de départ, solide et documentée, sur laquelle on peut ensuite construire.

Les autres switches du lab, **fed-a-capit** et **fed-a-infra**, servent à donner un contexte réseau crédible. Ils ne sont pas la cible principale de la supervision.

Les mots techniques, traduits

Terme	En clair
SNMP	Le langage que le switch utilise pour partager son état.
MIB	Le catalogue des informations disponibles sur l'équipement.
OID	L'adresse précise d'une information dans ce catalogue.
Exporter	Un petit service qui traduit le SNMP en un format lisible par Prometheus.
Prometheus	La base de données qui enregistre les mesures au fil du temps.
Grafana	L'outil qui transforme les mesures en graphiques.
Scrape	Une collecte automatique, à intervalle régulier.
Cible (target)	L'équipement ou le service que Prometheus interroge.
Dashboard	Le tableau de bord qui rassemble les graphiques.
Conteneur	Une mini-machine isolée qui fait tourner un service.
HTTP	Le protocole utilisé par les pages web et les API.

La chaîne de supervision

Le schéma est volontairement simple. Chaque outil fait une chose, et une seule :

```
Switch Cisco vIOS → SNMP → snmp-exporter → Prometheus → Grafana
```

Le switch répond aux questions. L'exporter traduit les réponses. Prometheus les stocke. Grafana les rend lisibles. Si un maillon ne fonctionne pas, le graphique ne s'affiche pas : c'est pourquoi j'ai vérifié la chaîne dans cet ordre.

Les ports utilisés sont eux aussi simples à retenir : SNMP sur UDP 161, l'exporter sur le port 9116, Prometheus sur le port 9090 et Grafana sur le port 3000. Dans le lab, tout reste dans le même réseau Docker.

1. Préparation

Avant de configurer quoi que ce soit, il faut un terrain de jeu. Un lab virtuel permet de tester, de casser, de recommencer, sans jamais mettre en danger le réseau réel. C'est la première brique du projet.

1.1 Pourquoi un lab virtuel ?

Un lab, c'est une répétition générale. On y retrouve les mêmes gestes que sur un équipement réel, mais dans un environnement isolé. Si une commande est mauvaise, on la corrige et on recommence. Si un conteneur tombe, on le relance. Le réseau de production, lui, ne bouge pas.

J'ai utilisé **Containerlab** pour décrire la topologie dans un fichier texte structuré (YAML), et **Docker** pour exécuter les conteneurs. Containerlab est le plan de montage ; Docker est l'atelier qui fait tourner les machines.

Le switch Cisco vIOS est la version virtuelle du système Cisco IOS : il se comporte comme un équipement réel, mais il tourne dans un conteneur. C'est ce qui permet de le configurer sans matériel physique.

Le lab contient six conteneurs : trois switches Cisco vIOS, un serveur Prometheus, un serveur Grafana et un snmp-exporter. Le switch supervisé s'appelle **fed-a-anp**. Les deux autres switches, **fed-a-capit** et **fed-a-infra**, complètent le décor et rendent la topologie plus réaliste.

1.2 Les outils, chacun son rôle

Outil	Rôle	Analogie
Containerlab	Décrit et démarre la topologie	Le plan de montage
Docker	Exécute les conteneurs	L'atelier
Cisco vIOS	Le switch à superviser	L'équipement sur le terrain
SNMP	Protocole de dialogue avec l'équipement	La langue commune
snmp-exporter	Traduit SNMP en métriques HTTP	Le traducteur
Prometheus	Collecte et stocke les métriques	Le carnet de relevés
Grafana	Affiche les graphiques	Le tableau de bord

Ces outils sont open source, largement utilisés et complémentaires. Prometheus et Grafana forment une paire classique en supervision ; SNMP est le standard historique des équipements réseau. Containerlab et Docker permettent de reproduire le lab sur n'importe quelle machine, sans matériel physique.

1.3 Le lab tourne

Première vérification : les conteneurs sont-ils bien démarrés ? La commande `containerlab inspect` affiche l'état de chaque équipement, son image et son adresse IP. C'est la photo d'ensemble du lab.

```
marwane@DESKTOP-MAR:~/anp-projet-mohammedia$ sudo containerlab inspect -t port.clab.yml
docker ps
22:35:38 INFO Parsing & checking topology file=port.clab.yml
```

Name	Kind/Image	State	IPv4/6 Address
clab-port-mohammedia-fed-a-anp	cisco_vios vrnetlab/cisco_vios:L2-20200929	running (healthy)	172.20.20.3 3fff:172:20:20::3
clab-port-mohammedia-fed-a-capit	cisco_vios vrnetlab/cisco_vios:L2-20200929	running (healthy)	172.20.20.4 3fff:172:20:20::4
clab-port-mohammedia-fed-a-infra	cisco_vios vrnetlab/cisco_vios:L2-20200929	running (healthy)	172.20.20.5 3fff:172:20:20::5
clab-port-mohammedia-grafana	linux grafana/grafana:latest	running	172.20.20.6 3fff:172:20:20::6
clab-port-mohammedia-prometheus	linux prom/prometheus:latest	running	172.20.20.7 3fff:172:20:20::7
clab-port-mohammedia-snmp-exporter	linux prom/snmp-exporter:latest	running	172.20.20.2 3fff:172:20:20::2

```
CONTAINER ID   IMAGE                              COMMAND                  CREATED         STATUS                   PORTS                                                NAMES
51aeeff5a801   vrnetlab/cisco_vios:L2-20200929    "uv run /launch.py -…"   5 minutes ago   Up 5 minutes (healthy)   22/tcp, 5000/tcp, 10000-10099/tcp, 161/udp           clab-port-m
ohammedia-fed-a-capit
5d25ac8b5f93   vrnetlab/cisco_vios:L2-20200929    "uv run /launch.py -…"   5 minutes ago   Up 5 minutes (healthy)   22/tcp, 5000/tcp, 10000-10099/tcp, 161/udp           clab-port-m
ohammedia-fed-a-infra
2e76c3d4916d   vrnetlab/cisco_vios:L2-20200929    "uv run /launch.py -…"   5 minutes ago   Up 5 minutes (healthy)   22/tcp, 5000/tcp, 10000-10099/tcp, 161/udp           clab-port-m
ohammedia-fed-a-anp
0f59e058a383   prom/prometheus:latest             "/bin/prometheus --c…"   5 minutes ago   Up 15 seconds            0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp          clab-port-m
ohammedia-prometheus
081328581d3f   grafana/grafana:latest             "/run.sh"                5 minutes ago   Up 5 minutes             0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp          clab-port-m
ohammedia-grafana
62e03a94413c   prom/snmp-exporter:latest          "/bin/snmp_exporter …"   5 minutes ago   Up 5 minutes             0.0.0.0:9116->9116/tcp, [::]:9116->9116/tcp          clab-port-m
ohammedia-snmp-exporter
marwane@DESKTOP-MAR:~/anp-projet-mohammedia$
```

Figure 1 — Les six conteneurs du lab, tous en running (healthy).

La table de Containerlab confirme que les six conteneurs sont en **running (healthy)**. On y lit aussi les adresses : fed-a-anp en 172.20.20.3, Prometheus en 172.20.20.7, Grafana en 172.20.20.6, snmp-exporter en 172.20.20.2. Cette table me sert de plan d'adressage pour la suite.

La commande `docker ps` complète cette vue : elle montre les conteneurs, leur image, leur statut et les ports exposés. C'est elle qui m'indique où joindre les interfaces web : 9090 pour Prometheus, 3000 pour Grafana, 9116 pour l'exporter.

```
marwane@DESKTOP-MAR:~/anp-projet-mohammedia$ docker ps
CONTAINER ID   IMAGE                              COMMAND                  CREATED          STATUS                     PORTS                                                NAMES
3757620a41eb   vrnetlab/cisco_vios:L2-20200929    "uv run /launch.py -…"   38 minutes ago   Up 38 minutes (healthy)    22/tcp, 5000/tcp, 10000-10099/tcp, 161/udp           clab-po
rt-mohammedia-fed-a-capit
57a3499e6eff   vrnetlab/cisco_vios:L2-20200929    "uv run /launch.py -…"   38 minutes ago   Up 38 minutes (healthy)    22/tcp, 5000/tcp, 10000-10099/tcp, 161/udp           clab-po
rt-mohammedia-fed-a-anp
785f27c52f3a   prom/prometheus:latest             "/bin/prometheus --c…"   38 minutes ago   Up 27 minutes              0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp          clab-po
rt-mohammedia-prometheus
a9259679f920   vrnetlab/cisco_vios:L2-20200929    "uv run /launch.py -…"   38 minutes ago   Up 38 minutes (unhealthy)  22/tcp, 5000/tcp, 10000-10099/tcp, 161/udp           clab-po
rt-mohammedia-fed-a-infra
29a4db7c12fd   grafana/grafana:latest             "/run.sh"                38 minutes ago   Up 38 minutes              0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp          clab-po
rt-mohammedia-grafana
4e87a7838b4c   prom/snmp-exporter:latest          "/bin/snmp_exporter …"   38 minutes ago   Up 38 minutes              0.0.0.0:9116->9116/tcp, [::]:9116->9116/tcp          clab-po
rt-mohammedia-snmp-exporter
marwane@DESKTOP-MAR:~/anp-projet-mohammedia$
```

Figure 2 — Les conteneurs Docker et les ports exposés.

Sur la capture `docker ps`, un conteneur apparaît en **unhealthy** (fed-a-infra). Cela ne veut pas dire qu'il est arrêté : il tourne, mais son test de santé échoue. C'est un signal à surveiller, et une bonne raison de ne pas se contenter d'un simple « le conteneur est lancé ». Pour la suite, le switch cible fed-a-anp est bien disponible, et la chaîne de supervision fonctionne. J'ai noté ce point comme une amélioration possible du lab.

> **À RETENIR**
>
> Un conteneur qui tourne n'est pas forcément un service qui répond. Docker distingue l'état (running) de la santé (healthy/unhealthy). En supervision, cette nuance évite de passer à côté d'un problème.

1.4 Le plan d’adressage

Rôle	Conteneur	Adresse
Switch supervisé	clab-port-mohammedia-fed-a-anp	172.20.20.3
Switch capitainerie	clab-port-mohammedia-fed-a-capit	172.20.20.4
Switch infrastructure	clab-port-mohammedia-fed-a-infra	172.20.20.5
Grafana	clab-port-mohammedia-grafana	172.20.20.6
Prometheus	clab-port-mohammedia-prometheus	172.20.20.7
snmp-exporter	clab-port-mohammedia-snmp-exporter	172.20.20.2

Tous ces équipements sont dans le réseau **172.20.20.0/24**. Le switch cible et l’exporter peuvent donc se joindre directement, sans passer par une route complexe. C’est une condition simple, mais indispensable, pour que la collecte SNMP fonctionne.

> **À RETENIR**
>
> Le plan d’adressage est la carte du lab. Le switch cible et l’exporter sont dans le même réseau : c’est cette proximité qui rend la collecte SNMP possible.

2. Configuration

La préparation est terminée. Place à la configuration : donner une adresse au switch, lui apprendre à répondre en SNMP, puis dire à Prometheus où et comment l'interroger.

2.1 Se connecter au switch

Dans le lab, la console du switch est accessible via Telnet, une connexion à distance en mode texte, sur le port 5000 publié par Containerlab. La commande suivante ouvre une session dans le conteneur, puis se connecte à la console :

```
docker exec -it clab-port-mohammedia-fed-a-anp telnet 127.0.0.1 5000
```

Une fois connecté, je vérifie deux choses : l'état des interfaces et la table de routage. C'est le b.a.-ba avant de parler de supervision.

```
marwane@DESKTOP-MAR:~/anp-projet-mohammedia$ docker exec -it clab-port-mohammedia-fed-a-anp telnet 127.0.0.1 5000
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.

fed-a-anp#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     172.20.20.3     YES manual up                    up
GigabitEthernet0/1     unassigned      YES unset  up                    up
GigabitEthernet0/2     unassigned      YES unset  up                    up
GigabitEthernet0/3     unassigned      YES unset  up                    up
GigabitEthernet1/0     unassigned      YES unset  up                    up
fed-a-anp#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
       a - application route
       + - replicated route, % - next hop override, p - overrides from PfR

Gateway of last resort is not set

      172.20.0.0/16 is variably subnetted, 2 subnets, 2 masks
C        172.20.20.0/24 is directly connected, GigabitEthernet0/0
L        172.20.20.3/32 is directly connected, GigabitEthernet0/0
fed-a-anp#
```

Figure 3 — Connexion au switch, état des interfaces et table de routage.

La commande `show ip interface brief` affiche GigabitEthernet0/0 avec l'adresse **172.20.20.3**, un statut **up** et un protocole **up**. Autrement dit : l'interface qui porte l'adresse est active. La commande `show ip route` confirme que le réseau **172.20.20.0/24** est directement connecté, avec une route locale pour l'adresse du switch.

Cette étape peut sembler basique, mais elle évite de chercher un problème SNMP là où il n'y en a pas. Si l'adresse est absente ou si l'interface est down, aucun outil de supervision ne pourra récupérer quoi que ce soit.

2.2 Activer SNMP sur le switch

SNMP est le langage que le switch va utiliser pour répondre aux questions de l'exporter. Pour faire simple : Prometheus demande « comment vont tes interfaces ? », le switch répond avec des chiffres. Encore faut-il lui donner la permission de répondre.

Dans le lab, j'ai activé SNMP en lecture seule avec les commandes suivantes :

```
enable
configure terminal
snmp-server community public RO
snmp-server location ANP-Mohammedia-Admin
snmp-server contact Support-IT
end
write memory
```

La communauté `public` est le « mot de passe » partagé pour lire les informations. **RO** signifie *read-only* : l'outil de supervision peut lire, mais pas modifier la configuration. Les champs `location` et `contact` ajoutent des informations utiles pour l'exploitation : où se trouve l'équipement et qui contacter.

Pour vérifier que la configuration est bien prise en compte, j'utilise :

```
show running-config | include snmp
```

```
fed-a-anp>
fed-a-anp>show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     172.20.20.3     YES manual up                    up
GigabitEthernet0/1     unassigned      YES unset  up                    up
GigabitEthernet0/2     unassigned      YES unset  up                    up
GigabitEthernet0/3     unassigned      YES unset  up                    up
GigabitEthernet1/0     unassigned      YES unset  up                    up
fed-a-anp>en
fed-a-anp#show running-config | include snmp
snmp-server community public RO
snmp-server location ANP-Mohammedia-Admin
snmp-server contact Support-IT
fed-a-anp#
```

Figure 4 — La configuration SNMP est présente et le switch est prêt à répondre.

La capture montre les trois lignes `snmp-server`. Le switch est donc prêt à parler SNMP. La commande `write memory`, absente de la capture mais utilisée juste après, permet de conserver la configuration après un redémarrage.

> **POINT DE VIGILANCE**
>
> En production, `public` n'est pas une configuration acceptable. Il faut utiliser SNMPv3 (authentification et chiffrement), une communauté complexe si SNMPv2 est imposé, des ACL pour limiter les sources autorisées, et un accès strictement en lecture seule. Le lab utilise `public` pour rester simple et reproductible ; ce choix ne doit pas sortir du lab. Et dans tous les cas, SNMP ne doit jamais être exposé sur Internet.

2.3 Déclarer la cible dans Prometheus

Prometheus ne parle pas SNMP nativement. Il a besoin d'un fichier de configuration pour savoir quoi interroger et à quelle fréquence. Ce fichier, `prometheus.yml`, est le carnet d'adresses de la supervision.

```
marwane@DESKTOP-MAR:~/anp-projet-mohammedia$ cat prometheus.yml
global:
  scrape_interval: 15s
  scrape_timeout: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'snmp'
    scrape_timeout: 15s
    static_configs:
      - targets:
          - 172.20.20.3
    metrics_path: /snmp
    params:
      module: [if_mib]
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
      - target_label: __address__
        replacement: snmp-exporter:9116
marwane@DESKTOP-MAR:~/anp-projet-mohammedia$
```

Figure 5 — Le fichier prometheus.yml, avec les deux cibles et le module SNMP.

Le fichier tient en quelques lignes, mais chaque ligne a son importance :

- `scrape_interval: 15s` : Prometheus pose une question toutes les 15 secondes.
- `scrape_timeout: 15s` : il attend au maximum 15 secondes avant de considérer que la collecte a échoué.
- `job_name: prometheus` : Prometheus se surveille lui-même, sur `localhost:9090`.
- `job_name: snmp` : la cible est le switch **172.20.20.3**, via le chemin `/snmp` et le module `if_mib`.
- `relabel_configs` : conserve l'adresse du switch dans le label `instance`, pour que les graphiques parlent du switch et non de l'exporter.

Le module `if_mib` est le catalogue des interfaces : statut, nom, compteurs de trafic. C'est lui qui permet de répondre à la question « les câbles sont-ils branchés ? » et « combien de trafic passe ? ».

Pourquoi 15 secondes ? C'est un bon compromis dans un lab : assez fréquent pour voir les variations, assez espacé pour ne pas surcharger l'équipement. En production, 30 ou 60 secondes suffisent souvent.

Dans les `relabel_configs`, la dernière règle remplace l'adresse de collecte par `snmp-exporter:9116`. Prometheus contacte donc l'exporter, qui contacte le switch. C'est un détour, mais c'est justement le rôle du traducteur.

2.4 Relier Grafana à Prometheus

Grafana ne stocke pas les données : il interroge Prometheus et dessine les résultats. Je l'ai donc connecté à Prometheus avec l'URL `http://prometheus:9090`, puis j'ai créé deux dashboards : **Statut des Interfaces** et **Bande Passante**.

Cette séparation est volontaire : Prometheus garde l'historique, Grafana se concentre sur la visualisation. On peut changer l'un sans casser l'autre.

> **À RETENIR**
>
> Grafana ne stocke pas les données : il interroge Prometheus. Si Prometheus s'arrête, les graphiques se vident, même si Grafana reste allumé.

3. Collecte

La configuration est en place. Il reste à prouver que les données circulent vraiment, du switch jusqu'à Prometheus. C'est l'étape que je préfère, parce qu'elle transforme une configuration en résultat concret.

3.1 Le rôle de snmp-exporter

snmp-exporter est un petit service qui écoute sur le port **9116**. On lui envoie une requête HTTP avec deux informations : l'adresse du switch et le module à interroger. Il se charge de poser les questions SNMP, puis renvoie les réponses dans un format que Prometheus comprend.

L'URL ressemble à ceci :

```
http://localhost:9116/snmp?target=172.20.20.3&module=if_mib
```

`target` est le switch à interroger. `module` est le catalogue de questions. `if_mib` correspond aux interfaces réseau. C'est simple, mais c'est toute la magie du traducteur.

3.2 Vérifier les données brutes

Avant de regarder un graphique, je vérifie la source. La commande `curl`, qui récupère le contenu d'une URL, interroge l'exporter et affiche les premières lignes de la réponse :

```
curl -s "http://localhost:9116/snmp?target=172.20.20.3&module=if_mib" | head -n 20
```

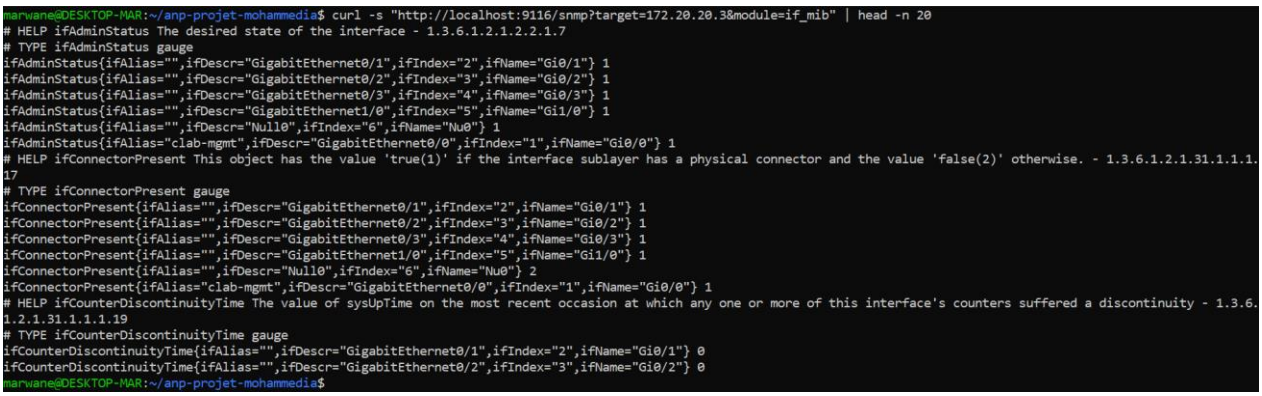


Figure 6 — Les métriques SNMP brutes, renvoyées par l'exporter en HTTP.

On y voit des métriques comme `ifAdminStatus`, `ifConnectorPresent` et `ifCounterDiscontinuityTime`, avec des labels `ifDescr`, `ifIndex` et `ifName`. Pour faire simple :

- `ifAdminStatus` indique l'état administratif de l'interface (1 = up).
- `ifConnectorPresent` indique si un connecteur physique est présent (1 = oui, 2 = non).
- `ifCounterDiscontinuityTime` indique si les compteurs ont été réinitialisés.
- `Null0` est une interface virtuelle du switch ; elle n'a pas de câble physique, ce qui explique sa valeur différente.

Cette réponse est brute, mais elle prouve l'essentiel : le switch répond, l'exporter traduit, et les métriques sont disponibles. Prometheus n'aura plus qu'à les stocker.

3.3 Vérifier la cible dans Prometheus

Direction l'interface web de Prometheus, dans **Status > Targets**. C'est la page qui montre si Prometheus arrive à joindre ses cibles.

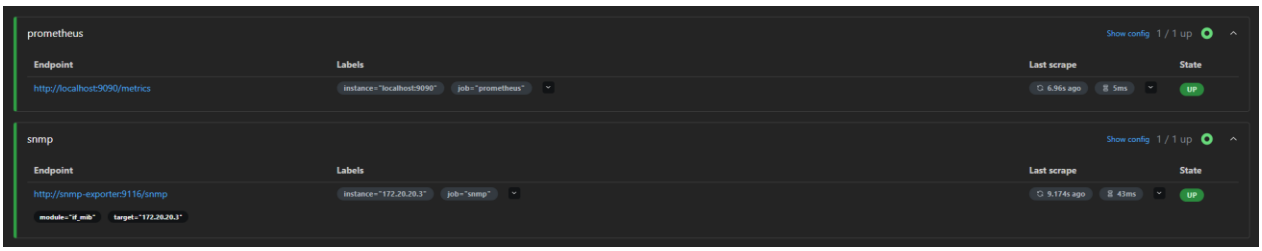


Figure 7 — Les deux cibles Prometheus sont UP, dont la cible SNMP du switch.

Les deux cibles sont **UP**. Pour la cible `snmp`, on lit l'endpoint `http://snmp-exporter:9116/snmp`, les labels `instance=172.20.20.3`, `job=snmp`, `module=if_mib` et `target=172.20.20.3`. La dernière collecte date de quelques secondes, et elle a pris 43 ms.

UP ne signifie pas seulement « le switch est allumé ». Cela signifie que Prometheus a réussi à contacter l'exporter, que l'exporter a réussi à interroger le switch, et que les métriques ont été récupérées. C'est la validation de toute la chaîne.

> **À RETENIR**
>
> Un graphique peut être joli et faux. La page Targets est la preuve que la donnée existe avant d'être dessinée.

3.4 Si ça ne marche pas : la checklist

Une chaîne de supervision comporte plusieurs maillons. Quand ça bloque, la bonne méthode est de les vérifier dans l'ordre, du plus simple au plus compliqué.

Étape	Question	Commande / vérification
1	Le switch répond-il ?	`ping 172.20.20.3` depuis le lab
2	L'exporter répond-il ?	`curl http://localhost:9116/snmp?target=...`
3	La communauté SNMP est-elle bonne ?	`show running-config | include snmp`
4	Le module est-il le bon ?	`module=if_mib` dans `prometheus.yml`
5	Les ports sont-ils accessibles ?	UDP 161 pour SNMP, TCP 9116 pour l'exporter
6	Que disent les logs ?	`docker logs clab-port-mohammedia-snmp-exporter`

La plupart du temps, le problème se trouve dans un maillon simple : une adresse, un port, un mot de passe ou un conteneur arrêté. Vérifier dans l'ordre évite de modifier dix choses à la fois.

4. Graphiques finaux

Les données arrivent. Il reste à les rendre lisibles. C'est le rôle de Grafana : transformer des séries de chiffres en deux dashboards compréhensibles par un ingénieur, un manager ou un recruteur non technique.

4.1 Dashboard 1 — État des interfaces

Le premier dashboard répond à une question simple : est-ce que les liens sont debout ? Chaque interface suivie est représentée par une ligne. Tant que la ligne reste stable, l'interface reste dans le même état.

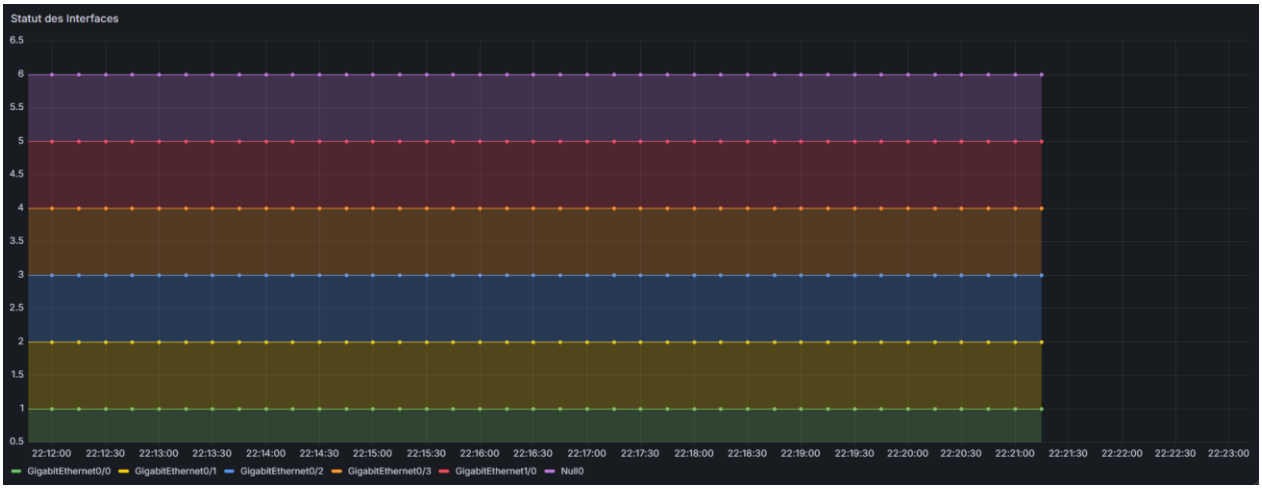


Figure 8 — Le dashboard Statut des Interfaces, avec une ligne par interface.

Sur cette capture, les lignes sont stables sur toute la période. Cela signifie que les interfaces suivies sont restées dans le même état : les câbles sont bien branchés, et aucun lien n'a décroché. C'est le premier réflexe en cas d'incident : vérifier si le problème vient du physique avant de chercher plus loin.

`Null0` apparaît dans la liste : c'est une interface virtuelle du switch, pas un câble. Sa présence est normale. Un dashboard utile sait aussi expliquer ce qu'il montre, pas seulement afficher des courbes.

Ce dashboard répond à la question : **« Est-ce que mes liens sont debout ? »**

> **À RETENIR**
>
> Une ligne stable signifie que l'interface reste dans le même état. Un décrochage invite à vérifier le câble, le port ou la configuration avant de chercher ailleurs.

4.2 Dashboard 2 — Bande passante

Le deuxième dashboard va un cran plus loin : il montre le trafic qui passe réellement sur les interfaces,

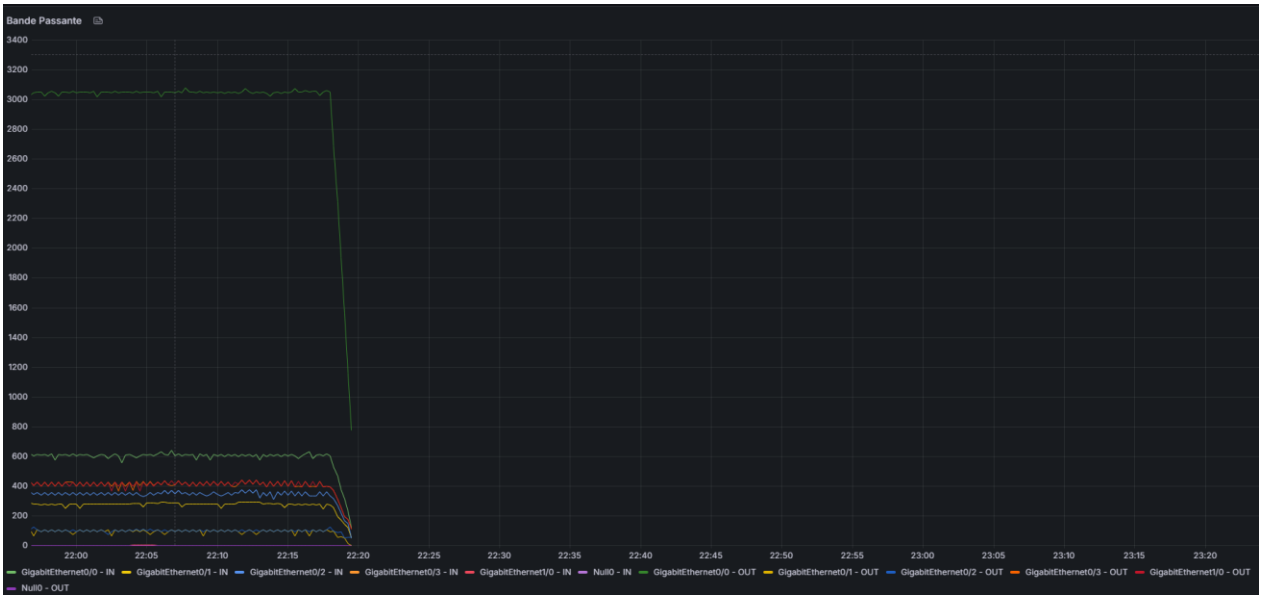


Figure 9 — Le dashboard Bande Passante, avec les courbes de trafic entrant et sortant.

Les courbes montent, descendent, respirent. C'est normal : le trafic réseau n'est jamais parfaitement plat. Ce qui compte, c'est la tendance, les pics et les décrochages. Une courbe qui sature en permanence peut annoncer un lien trop petit ; une chute brutale peut signaler un incident.

La chute à zéro en fin de graphique correspond à l'arrêt volontaire du lab. Ce n'est pas une panne : c'est la fin de la collecte. C'est même une preuve que le dashboard suit l'état réel de l'équipement.

Ce dashboard répond à la question : **« Est-ce que le réseau travaille normalement ? »**

4.3 Ce que ces graphiques changent

Avant	Après
Vérification manuelle, ponctuelle	Vue automatique toutes les 15 secondes
« Le switch est-il allumé ? »	« Les interfaces sont-elles up ? »
Aucun historique	Courbes et tendances dans le temps
Découverte d'une panne par appel utilisateur	Détection visuelle rapide

Le but n'est pas de remplacer l'ingénieur. C'est de lui donner la bonne information au bon moment, et de garder une trace pour comprendre ce qui s'est passé.

> **LA PROCHAINE ÉTAPE**
>
> Ce lab montre la chaîne technique, mais il ne déclenche pas encore d'alerte. Aujourd'hui, il faut ouvrir Grafana pour voir un problème. La suite logique serait d'ajouter Alertmanager ou les alertes Grafana pour être prévenu automatiquement.

Conclusion

Ce projet m'a permis de construire une chaîne de supervision de bout en bout, et pas seulement de configurer un outil. J'ai commencé par un lab, j'ai donné une adresse au switch, je lui ai appris à répondre en SNMP, j'ai vérifié les données brutes, puis je les ai vues dans Prometheus et dans Grafana. Chaque étape a été testée et documentée.

Ce que le projet démontre

- Le switch est joignable et correctement adressé.
- SNMP est activé en lecture seule et répond.
- L'exporter traduit les métriques sans erreur.
- Prometheus collecte la cible toutes les 15 secondes.
- Grafana affiche deux dashboards exploitables.

Compétences mobilisées

Domaine	Ce que j'ai pratiqué
Réseau	Cisco IOS, adressage IP, interfaces, routage connecté
Supervision	SNMP, MIB, exporter, Prometheus, Grafana
Infrastructure	Docker, Containerlab, Linux
Méthode	Vérification maillon par maillon, documentation, vulgarisation

Ce que j'ai appris

- **Un graphique n'est que la partie visible d'une chaîne.** Si la base est fragile, le graphique est fragile.
- **Un indicateur UP ne suffit pas.** Il faut savoir ce qu'il mesure et ce qu'il ne mesure pas.
- **La supervision la plus utile est celle qui répond à une question simple**, pas celle qui affiche le plus de courbes.

Limites et prochaines étapes

Le projet est volontairement limité à un lab. Pour passer en production, plusieurs étapes seraient nécessaires :

- Remplacer `public` par **SNMPv3**, avec authentification et chiffrement.
- Ajouter des **ACL** pour n'autoriser que les serveurs de supervision.
- Mettre en place des **alertes** (Alertmanager ou Grafana Alerting) pour être prévenu automatiquement.
- Déployer la même chaîne sur les **switches réels**, avec des dashboards par site.
- Définir une politique de **rétention des métriques** et sauvegarder les configurations.
- Intégrer les alertes à l'**outil de ticketing** de l'ANP.

> **SI JE DEVAIS CONTINUER DEMAIN**
>
> Je commencerais par les alertes : un dashboard qu'on doit ouvrir manuellement ne prévient personne. Ensuite, je passerais SNMPv3 et les ACL, avant tout déploiement réel.

Mot de la fin

Un réseau supervisé n'est pas un réseau qui ne tombe jamais. C'est un réseau qu'on voit tomber, et qu'on répare plus vite.

Ce projet m'a donné une conviction : la supervision n'est pas un outil réservé aux grands systèmes. Avec des outils open source, une méthode simple et un peu de rigueur, on peut rendre visible l'invisible et donner aux équipes le temps de réagir.

Remerciements

Je remercie l'équipe de l'Agence Nationale des Ports, au Port de Mohammedia, pour son accueil et pour le contexte concret de cette mission. Je remercie également mon encadrant Jawad Ait Oughlad pour ses conseils et pour m'avoir laissé tester, vérifier et documenter la démarche.

Annexe — Mémo technique

Cette annexe rassemble les commandes utilisées dans le rapport. Elle est pensée pour être copiée dans un README GitHub ou rejouée dans un lab.

Démarrer et vérifier le lab

```
sudo containerlab deploy -t port.clab.yml
sudo containerlab inspect -t port.clab.yml
docker ps
```

Se connecter au switch

```
docker exec -it clab-port-mohammedia-fed-a-anp telnet 127.0.0.1 5000
```

Configurer SNMP

```
enable
configure terminal
snmp-server community public RO
snmp-server location ANP-Mohammedia-Admin
snmp-server contact Support-IT
end
write memory
```

Vérifier SNMP

```
show running-config | include snmp
```

Tester l'exporter

```
curl -s "http://localhost:9116/snmp?target=172.20.20.3&module=if_mib" | head -n 20
```

Consulter les logs

```
docker logs clab-port-mohammedia-snmp-exporter
```

Extrait de prometheus.yml

```
global:
  scrape_interval: 15s
  scrape_timeout: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'snmp'
    scrape_timeout: 15s
    static_configs:
      - targets:
          - 172.20.20.3
    metrics_path: /snmp
    params:
      module: [if_mib]
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
      - target_label: __address__
        replacement: snmp-exporter:9116
```

Point de sécurité

En production, remplacer `public` par SNMPv3, ajouter des ACL et ne jamais exposer SNMP sur Internet. Les fichiers principaux du projet sont `port.clab.yml` pour la topologie et `prometheus.yml` pour les cibles. Les dashboards Grafana sont exportables en JSON et peuvent être versionnés avec le reste du projet.

— Fin du rapport —